Tracing, Debugging & Runtime Analysis

Zephyr RTOS Intermediate Course





Quick Recap - Lectures 1-5



- **Scheduling:** priorities, preemption, latency, starvation
- **Synchronization:** races, mutexes, semaphores, deadlocks
- **Deferred execution:** timers, workqueues, `k_poll`
- **Communication:** message queues, FIFOs, pipes, zbus
- **Reliability:** stack usage, resource limits, watchdogs



Today's Agenda



- Runtime evidence and investigation workflow
- Structured logging and diagnostic overhead
- Thread-state inspection
- GDB on real hardware and `native_sim`
- CTF tracing on `native_sim`
- Babeltrace and optional Trace Compass visualization
- Deadlock and scheduling-delay demonstrations



Runtime Evidence



Runtime Failure Categories



Symptom	Possible Zephyr behavior
No progress	Thread pending, deadlock, empty queue
Late result	Runnable thread delayed or preempted
Wrong result	Race, lifetime error, missing synchronization
Reset or fault	Stack failure, invalid access, watchdog
Degradation over time	Queue growth or resource exhaustion

- One symptom can have several causes
- The first task is separating those causes



Response-Time Components




```
 event
   v
waiting -> ready -> running -> completed
   |         |         |
 object   scheduler  execution and
 delay     delay      preemption time
```

```
response time = waiting + scheduler delay + execution time
```

- **Pending:** cannot run until a dependency changes
- **Ready:** can run but has not been selected
- **Running:** currently executing on a CPU



Evidence Types



Evidence	Question answered
Structured logs	What application operation progressed?
Counters and latency	Which guarantee failed?
Shell snapshot	What is each thread doing now?
GDB	Which call and object define this state?
Scheduler trace	How did execution reach this state?

- Evidence sources complement each other
- No single source explains every concurrency failure



Investigation Workflow



- Define the failed guarantee
- Add a measurable detector
- Reproduce with controlled input
- Capture the smallest useful evidence set
- Explain one complete execution path
- Change one cause
- Repeat the original measurement
- **A correction is proven by the original detector**

Diagnostic Overhead

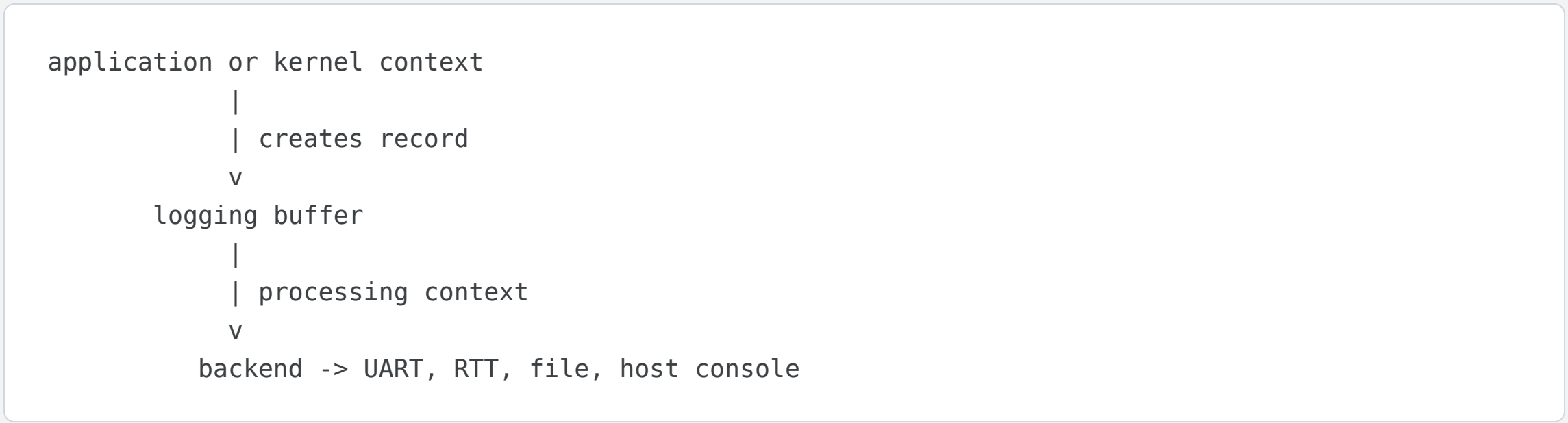
Diagnostic feature	Runtime effect
Logging	CPU, stack, buffer RAM, output bandwidth
Kernel shell	Extra thread, stack, transport activity
Runtime statistics	Scheduler accounting
Tracing	Event generation, bandwidth, storage
GDB	Stops or changes target execution

- Compare timing with a quieter diagnostic build
- Validate final behavior on the real target



Logging Strategy

Zephyr Logging Architecture



- A log call creates a structured record
- A backend decides where that record appears
- Processing mode decides where formatting occurs

Structured Event Logging

```
LOG_MODULE_REGISTER(worker, CONFIG_APP_LOG_LEVEL);

LOG_INF("event_ready seq=%u queue=%u",
        msg.seq, k_msgq_num_used_get(&sensor_q));

LOG_INF("event_done seq=%u latency_ms=%u result=%d",
        msg.seq, latency_ms, result);
```

- Give every operation a stable identifier
- Record state needed to explain success or failure
- Keep event names consistent across modules



Filtering and Rate Limiting




```
if (latency_ms > DEADLINE_MS) {
    deadline_misses++;
    LOG_WRN_RATELIMIT(
        "deadline_miss count=%u seq=%u latency_ms=%u",
        deadline_misses, msg.seq, latency_ms);
}
```

```
CONFIG_LOG_RATELIMIT=y
CONFIG_LOG_RATELIMIT_INTERVAL_MS=1000
```

- Count every failure
- Rate-limit repeated output
- Enable detailed logs only for relevant modules



Deferred Logging and Buffer Pressure



Mode	Caller behavior
Deferred	Stores a record for later processing
Immediate	Formats and sends from the caller's context

- A full buffer can drop records
- Overflow mode can discard older pending records
- Thread blocking can preserve logs but change scheduling
- Missing output does not always mean missing execution



Runtime Inspection

Kernel Thread Inspection

```
CONFIG_SHELL=y
CONFIG_KERNEL_SHELL=y
CONFIG_SHELL_BACKEND_SERIAL=y
CONFIG_THREAD_MONITOR=y
CONFIG_THREAD_NAME=y
CONFIG_SHELL_LOG_BACKEND=n
```

```
uart:~$ kernel thread list
uart:~$ kernel thread stacks
```

- `list` reports priority, state, timeout, and entry address
- `stacks` reports high-water usage where the architecture supports it



Interpreting Thread State




```
producer   priority: 5   state: sleeping
consumer   priority: 4   state: pending
*shell     priority: 14  state: queued
logging    priority: 14  state: pending
```

- `sleeping`: waiting for a timeout
- `pending`: blocked on a kernel object
- `queued`: ready for scheduler selection
- `*`: current thread producing the snapshot
- **High priority cannot make a pending thread runnable**



Runtime Statistics and Thread Analyzer




```
CONFIG_THREAD_RUNTIME_STATS=y
```

- Runtime statistics accumulate thread execution cycles
- They show CPU distribution over an interval
- They do not show individual context-switch order

```
CONFIG_THREAD_ANALYZER=y
```

- Adds stack and runtime reporting on supported targets
- **Zephyr 4.4 excludes `ARCH_POSIX`; it is unavailable on `native_sim`**



Snapshot Limitations



Observation	What it does not prove
Thread is pending	Which object blocks it
Thread is queued	How long it waited ready
High CPU percentage	Exact preemption order
Queue is full	Why the consumer stopped
Stack high-water mark	Current call depth

- A snapshot describes one instant
- Use GDB when a failure leaves stable object state
- Use tracing when scheduler history matters



Stable-State Debugging



GDB works well for:

- Deadlocks and permanently blocked threads
- Fault handlers and invalid memory state
- Inspecting queues, mutexes, semaphores, and counters
- Backtraces from known failure points

GDB is weaker for:

- Short races
- Deadline analysis
- Original interrupt and preemption timing

GDB on Real Hardware

```
CONFIG_DEBUG_THREAD_INFO=y
```

```
west build -b <board> app -p always
west debug
# Or attach without loading a new image:
west attach
```

- `west debug` selects the board's configured debug runner
- Use `west attach` to inspect an already-running image
- RTOS thread awareness depends on the runner and debug server
- Use the exact `build/zephyr/zephyr.elf` from the target



The Same Investigation on `native_sim`




```
west build -b native_sim app -p always
gdb build/zephyr/zephyr.exe
```

```
(gdb) break health_check
(gdb) run
(gdb) info threads
(gdb) thread 3
(gdb) backtrace
(gdb) frame 2
(gdb) info locals
```

- `native_sim` is a normal host executable with debug symbols
- It provides repeatable source and object inspection without a probe
- Confirm timing-dependent corrections on the real target



DEMO 1: Mutex Deadlock



Zephyr Tracing



When Scheduler History Is Required

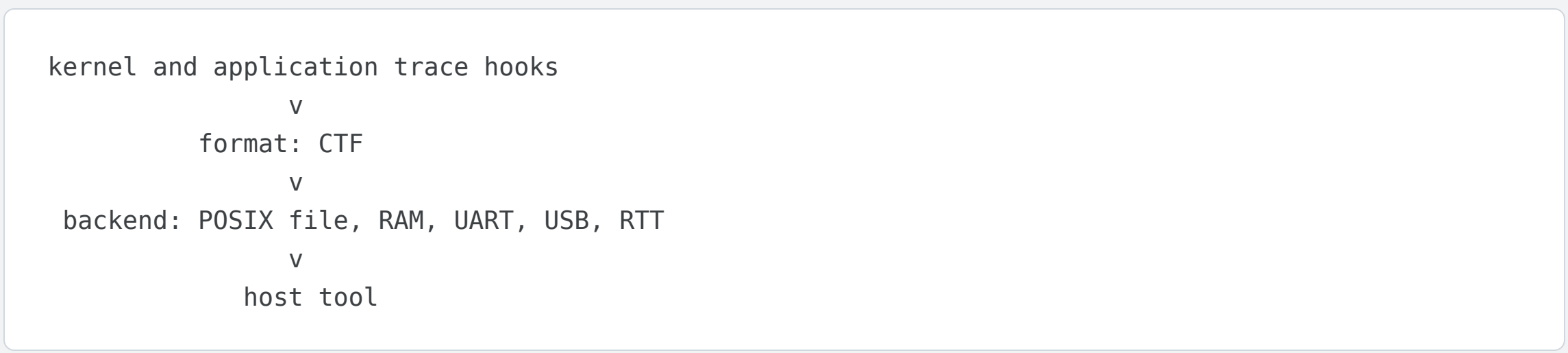



```
event_ready seq=42
event_done  seq=42 latency_ms=63
```

Possible explanations:

- Worker remained pending on a dependency
- Worker became ready but another thread ran
- Worker ran and was preempted
- Worker executed too much work
- **A snapshot after completion cannot separate these cases**

Zephyr Tracing Architecture



- Trace hooks define observable events
- The format defines event encoding
- The backend stores or transports encoded data
- Format and backend are separate choices



CTF Capture and Babeltrace




```
CONFIG_TRACING=y
CONFIG_TRACING_CTF=y
CONFIG_TRACING_SYNC=y
CONFIG_TRACING_BACKEND_POSIX=y
CONFIG_TRACING_PACKET_MAX_SIZE=64
```

```
mkdir -p data
cp $ZEPHYR_BASE/subsys/tracing/ctf/tsdl/metadata data/
./build/zephyr/zephyr.exe -trace-file=data/channel0_0
babeltrace2 data/
```

- `metadata` defines the Zephyr event schema
- `channel0_0` contains binary event records

Application and Scheduler Events

```
sys_trace_named_event("event_ready", msg.seq, queue_depth);
sys_trace_named_event("event_done", msg.seq, latency_ms);
```

```
named_event: event_ready seq=42
thread_ready: worker
thread_switched_out: producer
thread_switched_in: cpu_load
thread_switched_out: cpu_load
thread_switched_in: worker
named_event: event_done seq=42
```

- Named events identify the operation
- Scheduler events explain when its thread could execute



DEMO 2: Scheduling Delay Trace



Selecting Runtime Evidence



Failure	Start with	Escalate to
Incorrect result	Logs and counters	Trace or targeted watchpoint
Stable deadlock	Shell snapshot	GDB backtraces and objects
Missed deadline	Structured latency logs	Scheduler trace
CPU imbalance	Runtime statistics	Trace selected intervals
Fault or reset	Fatal output	GDB and preserved crash data

- Select evidence from the failure behavior
- Do not enable every diagnostic feature by default



Homework



- **Goal:** explain and reduce a Zephyr scheduling delay
- **Tasks:**
  - Run the provided producer-consumer application from branch `l6-homework`
  - Define and detect its response-time guarantee
  - Add sequence-based logs and deadline-miss counting
  - Inspect thread priorities and states with the kernel shell
  - Capture CTF and decode it with Babeltrace
  - Apply one correction and repeat the measurement
- Push tag: `l6-task1`

Thank You





Resources



1. Logging
2. Kernel Shell
3. Thread Runtime Statistics
4. Thread Analyzer
5. Debugging
6. Building, Flashing and Debugging
7. Native Simulator
8. Tracing
9. Tracing Sample
10. Babeltrace
11. Trace Compass